PRISMA OS: o que melhorar no visual, nas automações e na engenharia

Diagnóstico com evidência do código, direção visual com tokens e maquete, lista de automações e plano em fases. Uma fase por pedido.

Revisão	11/11/2026
Código	github.com/Gabrielnassa/PRISMA-OS · branch claude/session-vagl59 · commit 14d0c57 (04/11/2026)
Telas	instância do Codespace: Início, Clientes, página do cliente, Criar, Conteúdos, Aprovações, Custos, Integrações
Seções	1 Resumo · 2 Diagnóstico do visual · 3 Direção visual · 4 Tela por tela · 5 Automações do produto · 6 Automações de engenharia · 7 Outros pontos · 8 Plano por fases · 9 Para o Claude Code
Limites	testes e build não foram executados; alterações locais não enviadas ao GitHub não foram lidas; os layouts das peças geradas ficaram fora (aqui, "visual" é a interface do sistema)

1 Resumo

A base técnica está bem resolvida: isolamento por cliente também no banco (RLS), fila com idempotência, versões imutáveis, aprovação presa ao hash da versão, 28 arquivos de teste e 6 roteiros de navegador. O que destoa é a camada visível e a falta de automação em volta. Cinco pontos explicam quase tudo.

1. **O visual veio de uma página de marketing.** `docs/design/DESIGN-referencia.md` descreve o site de outro produto (Huly): hero, feixe aurora, botão "Sign Up". Essa linguagem de vitrine foi aplicada a uma ferramenta de trabalho. O resultado é o conjunto que mais se vê em painéis gerados por IA: fundo quase preto, azul de destaque, tudo em pílula, um cartão com borda para cada bloco, números gigantes.
2. **Não há sistema de medidas.** São 23 tamanhos de fonte, 127 usos com meio pixel (12,5 px, 13,5 px) e 355 valores avulsos entre colchetes nas classes. Interface feita com cuidado usa cinco ou seis tamanhos e não sai deles.
3. **As telas contam em vez de orientar.** O Início abre com seis contadores do mesmo peso (no seu Codespace: 1, 0, 0, 0, 0 e US$ 0,00). O bloco que de fato ajuda, o mês de cada cliente, vem depois dos contadores e de dois cartões que, vazios, só dizem "nada aqui".
4. **Quase toda automação depende de um clique.** A fila executa cinco tipos de tarefa (geração, publicação agendada, uma leitura de métricas, webhooks, e-mails) e uma limpeza de hora em hora. Não existe agendador recorrente: lembrete, importação do Drive, linha editorial e relatório do mês só acontecem quando alguém lembra de clicar.
5. **O repositório não tem rede de proteção automática.** Não existe `.github/` (nenhum CI), nem formatador, nem atualização de dependências, nem deploy automatizado. O único branch é `claude/session-vagl59`.

Ordem de execução

FASE	ENTREGA	POR QUE NESTA ORDEM
0 · Base	branch `main`, CI, formatador, dados de demonstração, capturas automáticas	rede de proteção e o "antes" para comparar
1 · Fundamentos visuais	tokens, fontes, componentes-base, barra lateral e barra de topo, trava no CI	todas as telas herdam
2 · Telas	Início, listas, formulários, peça, calendário, login, estados vazios	aplica a direção tela a tela
3 · Automações I	agendador, telas Automações e Fila, avisos que faltam	barato e de efeito imediato
4 · Automações II	Drive periódico, métricas com histórico, saúde das conexões, resumo semanal, e-mail em HTML	depende do agendador
5 · Automações III	linha editorial, relatório, pautas recorrentes, agendar ao aprovar, API de entrada	tocam IA ou publicação: nascem desligadas
6 · Operação	imagem Docker, deploy com retorno automático, monitoramento, restauração testada	antes de pôr cliente real em produção

2 Por que a interface parece feita por IA

Nenhum item abaixo é grave sozinho. O efeito vem da soma, e quase todos saem do mesmo lugar: os componentes de `src/components/ui.tsx`, desenhados a partir da referência de marketing. Os números são contagens no código do commit revisado.

#	SINAL	ONDE ESTÁ	MEDIDA
1	**Título de página em fonte de vitrine**	`PageHeader`: Sora, 30 a 36 px, tracking −0,04em	todas as telas do app; 14 usos de `font-display`
2	**Botões, etiquetas, busca e navegação em pílula**	`buttonClass`, `Badge`, `shell/sidebar.tsx`	48 usos de `rounded-full`
3	**Seletores nativos e botão "Filtrar"**	`Select` com `appearance-auto`	46 seletores; a seta é a do sistema operacional, dentro de uma interface escura sob medida
4	**Aviso em cartão com barra colorida à esquerda**	`Alert`	19 usos; "IA não conectada" aparece no aviso e de novo no rodapé da barra lateral
5	**Contadores gigantes**	`Stat` e `StatGroup`: Sora 32 px	11 usos; Início e Custos abrem com eles
6	**Um cartão com borda para cada bloco**	`Card`	85 usos; caixas com borda dentro do cartão em `editor.tsx` (prévia, aviso, bloco Imagem) e em `brand-form.tsx`
7	**Navegação sem hierarquia**	`shell/sidebar.tsx`	15 itens do mesmo peso; nenhum mostra quantas pendências tem
8	**Muitas cores de estado**	`TONES`, `DOTS`, `PIPELINE_BAR`	vermelho 48, verde 40, âmbar 36 e azul-céu 11 usos, além do íris (52) e do coral
9	**Tamanhos de fonte avulsos**	classes `text-[Npx]`	23 tamanhos, 295 usos, 127 com meio pixel
10	**Caixas tracejadas de "nada aqui"**	`EmptyState`	19 usos; com o sistema vazio, cada tela é um título grande e uma caixa
11	**Enfeites**	`.page-enter`, brilho no hover do botão principal, selo "OS" no logotipo, `(auth)/auth-shell.tsx`	animação de entrada em cada página; frase de vitrine e print do produto na tela de login

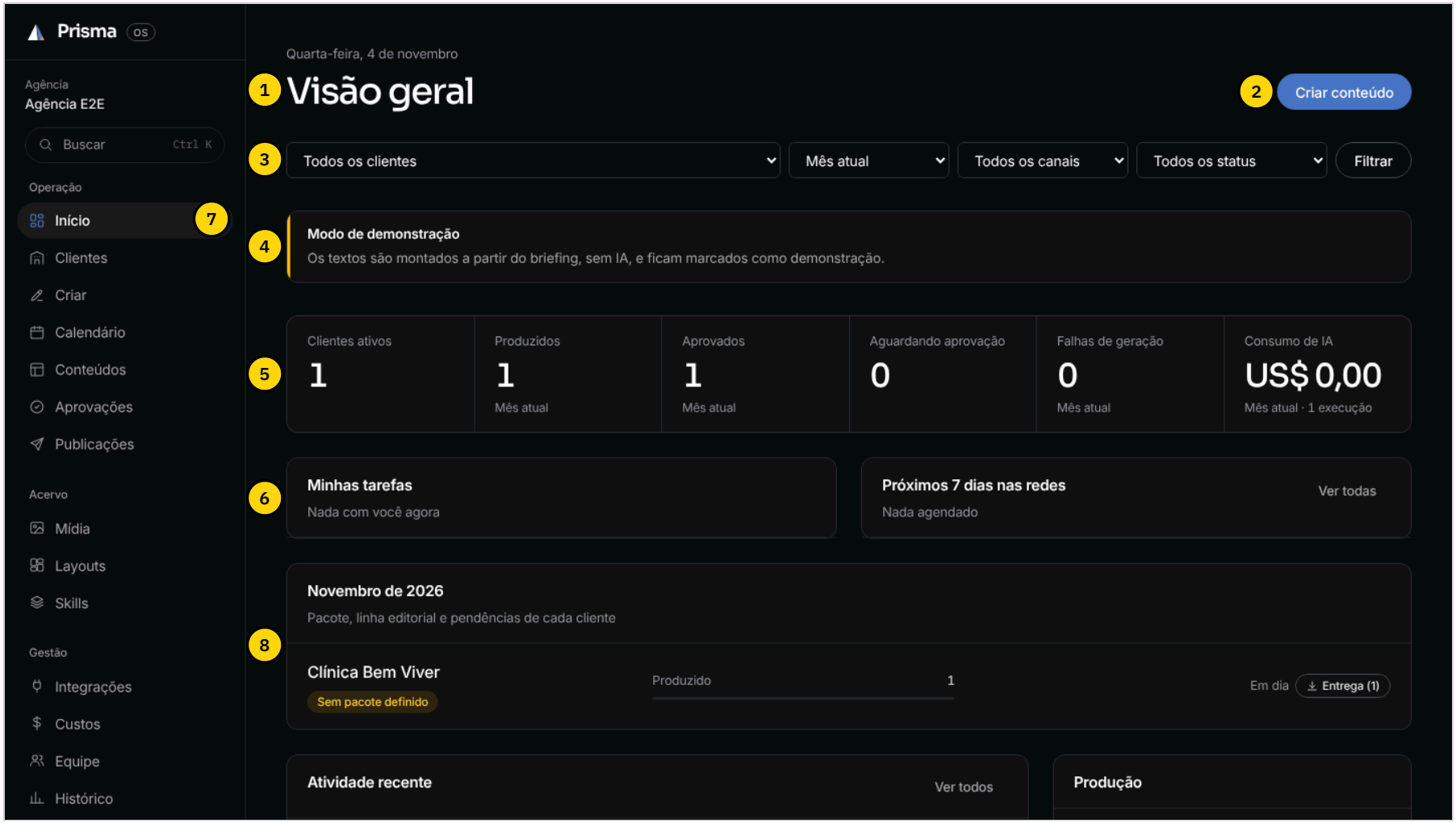
Defeitos de layout encontrados no caminho

- Custos, a 1440 px de largura: a tabela "Por skill" fica cortada. As tabelas têm `min-w-[640px]` e vão para duas colunas a partir de 1280 px.
- Custos: valores em dólar com duas e com quatro casas na mesma tela (US$ 0,00 e US$ 0,0000).
- Um único esqueleto de carregamento, no formato do Início, serve todas as rotas (`(app)/loading.tsx`).
- A página que o cliente abre para aprovar (`aprovar/[token]/page.tsx`) também é quase preta e usa mais um raio (`rounded-[10px]`).

O que já está certo e fica

- Contraste de texto pensado para o AA, foco visível e respeito a `prefers-reduced-motion`.
- Números tabulares, diálogo de confirmação próprio, avisos de resultado.
- Busca com Ctrl+K e filtros guardados na URL.
- Tabela de conteúdos que vira cartões no celular.
- Textos da interface curtos, diretos e em português.

Figura 1 · O Início hoje



1 Título em Sora 36 px, escala de página de marketing.

2 Botão principal em pílula azul; "Filtrar", também em pílula, logo abaixo.

3 Quatro seletores nativos e um botão só para aplicar o filtro.

4 Aviso em cartão com barra colorida; o mesmo estado se repete no rodapé da barra lateral.

5 Seis contadores do mesmo peso em Sora 32 px. Aqui valem 1, 1, 1, 0, 0 e US$ 0,00.

6 Dois cartões inteiros para dizer "nada com você agora" e "nada agendado".

7 Navegação em pílulas: 15 itens do mesmo peso, sem contador de pendência.

8 O bloco mais útil da tela, o mês de cada cliente, só aparece depois de tudo isso.

Captura do próprio repositório (`docs/img/07-dashboard.png`, recortada). A instância do Codespace mostra a mesma estrutura.

3 Direção visual: ferramenta de trabalho, não vitrine

O alvo é a interface de quem passa o dia dentro dela: densa, calma e previsível. Para calibrar o olho, sem copiar tela, marca ou logotipo de ninguém: Linear, GitHub, o painel da Stripe e o da Vercel.

3.1 Sete regras

1. **A tipografia faz a hierarquia.** Caixa com borda só quando separa objetos de verdade.
2. **Cinco tamanhos de fonte.** Fora deles, o CI recusa.
3. **Cor é sinal.** Neutro por padrão; vermelho e âmbar só para o que pede ação; um acento só.
4. **Retângulos discretos.** Raio de 4 px (6 px em menu e diálogo), linha fina, sem sombra, sem degradê, sem pílula.
5. **Tabela e lista antes de cartão.** Linha de 34 px, cabeçalho fixo, número alinhado à direita.
6. **Dado em fonte mono.** Datas, quantidades, valores, versões e identificadores.
7. **Toda tela começa pelo que precisa de ação.** Contador vem depois, em uma linha.

3.2 Tipografia

IBM Plex Sans na interface e IBM Plex Mono nos dados. O Mono já está no projeto; falta instalar `@fontsource/ibm-plex-sans`. A Sora sai (só `src/app/layout.tsx` a carrega). As fontes das peças, Inter incluída, não mudam. Alternativa aceitável: pilha do sistema (`system-ui`) na interface, com o Plex Mono nos dados.

TOKEN	TAMANHO / LINHA	PESO	USO
`text-xs`	11 / 16 px	500, maiúsculas, +0,05em	cabeçalho de tabela, rótulo de grupo
`text-sm`	12 / 16 px	400	texto secundário, dica, estado
`text-base`	13 / 20 px	400 e 500; 600 em título de seção	corpo: tabelas, formulários, navegação
`text-md`	15 / 22 px	600	título da página (na barra de topo) e de diálogo
`text-lg`	20 / 28 px	600	só onde não há barra de topo: login e páginas do cliente
`font-mono`	12 / 16 px	400 e 500	dados

3.3 Cores

Tokens com nome de função, sempre em par (escuro e claro). O botão principal passa a ser neutro invertido: fundo `--text`, texto `--bg`. O íris sai dos botões e fica no logotipo, no foco e nos links. Contraste calculado sobre `--bg`; todos os tons informativos também passam de 4,5:1 sobre `--surface-2`.

TOKEN	ESCURO	CLARO	USO	CONTRASTE (ESCURO / CLARO)
`--bg`	`#0f1011`	`#ffffff`	fundo da página	—
`--surface`	`#151618`	`#f6f6f7`	barra lateral, linha em foco	—
`--surface-2`	`#1c1d20`	`#eeeef0`	hover, item ativo	—
`--border`	`#26282c`	`#e2e3e6`	linhas e divisórias	—
`--border-control`	`#62656c`	`#878a91`	contorno de campo e de botão secundário	3,3:1 / 3,5:1
`--text`	`#e8e9eb`	`#17181a`	texto	15,7:1 / 17,8:1
`--muted`	`#9b9ea4`	`#5d6067`	texto secundário, rótulos	7,1:1 / 6,3:1
`--faint`	`#6f7278`	`#8a8d94`	só placeholder, desativado e ícone decorativo	4,0:1 / 3,3:1
`--accent`	`#8fb0ee`	`#2f56b0`	link, foco, seleção	8,7:1 / 6,8:1
`--danger`	`#f2686c`	`#c62f36`	falha	6,3:1 / 5,4:1
`--warn`	`#e2a336`	`#8f5f00`	atraso, pendência antiga	8,6:1 / 5,5:1
`--ok`	`#4cb782`	`#1c7a4f`	ponto de estado e barra completa	7,6:1 / 5,3:1

3.4 Forma e espaço

Raio	4 px em botão, campo, etiqueta e item de navegação; 6 px em menu, diálogo e aviso de resultado; círculo só em avatar, ponto de estado e interruptor
Altura de controle	28 px em barras e tabelas, 32 px em formulários; no celular, alvo de toque de 44 px
Tabela	linha de 34 px, célula com 10 px nas laterais, divisória de 1 px, sem moldura
Estrutura	grade de 4 px; barra lateral de 216 px; barra de topo de 48 px; margem de página de 28 px
Sombra	só em menu, diálogo e aviso de resultado

3.5 Ponto de partida para `src/app/globals.css`

```
@theme {
  --font-sans: "IBM Plex Sans", ui-sans-serif, system-ui, sans-serif;
  --font-mono: "IBM Plex Mono", ui-monospace, Menlo, monospace;
  --text-xs: 11px;    --text-xs--line-height: 16px;
  --text-sm: 12px;    --text-sm--line-height: 16px;
  --text-base: 13px;  --text-base--line-height: 20px;
  --text-md: 15px;    --text-md--line-height: 22px;
  --text-lg: 20px;    --text-lg--line-height: 28px;
  --radius-sm: 4px;   --radius-md: 6px;
  --color-bg: var(--bg);          --color-surface: var(--surface);   --color-surface-2: var(--surface-2);
  --color-line: var(--border);    --color-control: var(--border-control);
  --color-fg: var(--text);        --color-muted: var(--muted);       --color-faint: var(--faint);
  --color-accent: var(--accent);  --color-danger: var(--danger);     --color-warn: var(--warn);   --color-ok:
var(--ok);
}
:root {
  --bg: #0f1011; --surface: #151618; --surface-2: #1c1d20; --border: #26282c; --border-control: #62656c;
  --text: #e8e9eb; --muted: #9b9ea4; --faint: #6f7278;
  --accent: #8fb0ee; --danger: #f2686c; --warn: #e2a336; --ok: #4cb782;
}
:root[data-theme="light"] {
  --bg: #ffffff; --surface: #f6f6f7; --surface-2: #eeeef0; --border: #e2e3e6; --border-control: #878a91;
  --text: #17181a; --muted: #5d6067; --faint: #8a8d94;
  --accent: #2f56b0; --danger: #c62f36; --warn: #8f5f00; --ok: #1c7a4f;
}
```

Sintaxe do Tailwind 4 (o projeto usa a 4.3). Conferir na documentação da versão instalada antes de aplicar, como o `AGENTS.md` já pede para o Next.

3.6 Trava contra regressão

Criar `scripts/check-design.mjs` e chamá-lo no `npm run lint`. Ele recusa: `text-[`, `font-display`, `rounded-full` fora de avatar, ponto de estado e interruptor, cor direta do Tailwind (`sky`, `emerald`, `amber`, `red`) ou hexadecimal em classe dentro de `src/app` e `src/components`. Sem a trava, a sessão seguinte volta a espalhar valores avulsos.

Também nesta etapa: trocar `docs/design/DESIGN-referencia.md` por `docs/design/SISTEMA.md` com as regras e os tokens acima; tirar o selo "OS" do logotipo (o prisma fica); remover os ícones sem uso em `src/components/icons.tsx` (`strategy`, `sites`, `ads`, `chevron`, `plus`; `automations` passa a ser usado na tela nova).

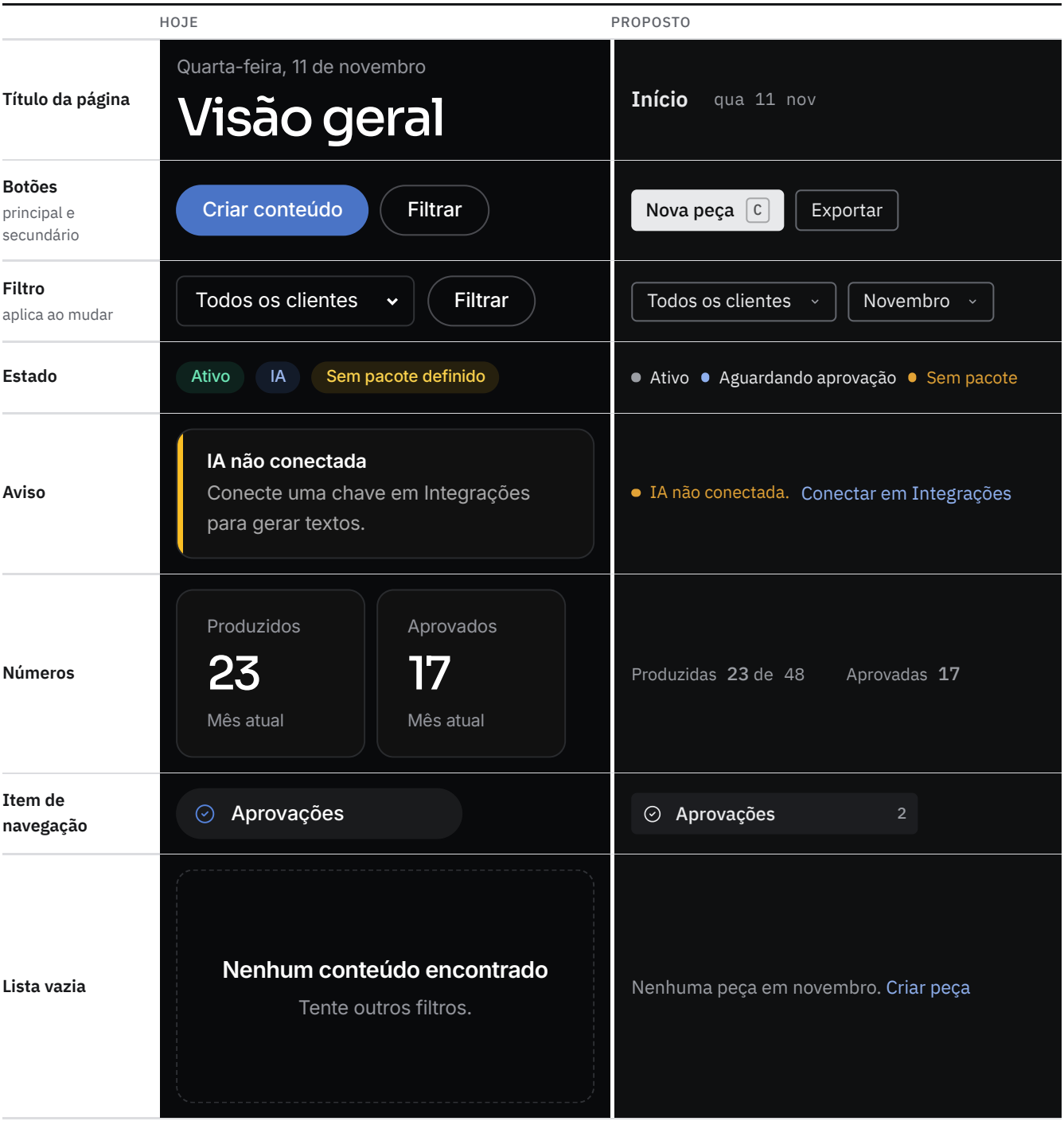
3.7 Componentes

Uma regra por componente. A Figura 2 mostra o antes e o depois de cada um.

COMPONENTE	REGRA
Botão	Principal neutro invertido, um por tela. Secundário com contorno. Fantasma para ação de linha. Perigo só dentro do diálogo de confirmação.
Campo e seleção	Contorno `--border-control`, seta própria. A lista de clientes vira campo com busca. Área de texto cresce com o conteúdo a partir de duas linhas.
Filtros	Aplicam ao mudar e continuam na URL. O botão "Filtrar" sai.
Estado	Ponto de 6 px e texto. Neutro para o que é normal (em revisão, aprovada, agendada); âmbar e vermelho só para atraso e falha. Sem fundo colorido.

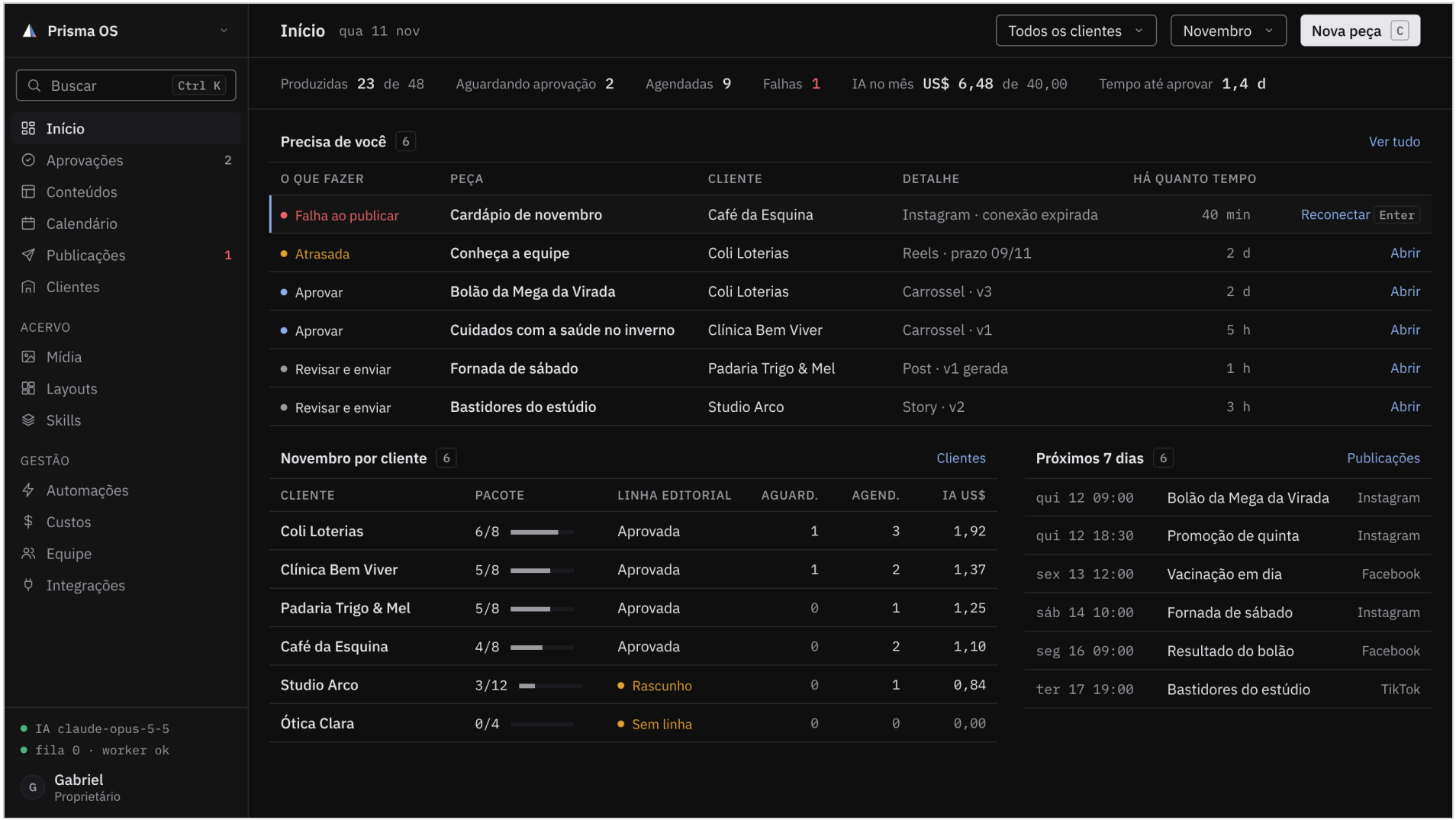
COMPONENTE	REGRA
Aviso	Uma linha, sem cartão e sem barra lateral. Estado permanente, como IA não conectada, aparece uma vez só, no rodapé da barra lateral.
Tabela	Um componente único: cabeçalho fixo, ordenação, linha inteira clicável, seleção em lote, colunas por prioridade no celular. Sem `min-w` fixo.
Página	Barra de topo de 48 px com título, contexto, filtros e ação principal. Sem título de 36 px e sem animação de entrada.
Carregamento	Um esqueleto por tipo de tela: lista, formulário e peça.

Figura 2 · Mesmos componentes, antes e depois



Coluna "Hoje": valores de `src/components/ui.tsx`. Coluna "Proposto": tokens da seção 3. As duas na mesma escala, a 90% do tamanho real. A linha branca à esquerda marca a coluna proposta.

Figura 3 · Direção proposta para o Início



Maquete, não captura. Dados ilustrativos. Mesma informação da Figura 1, reordenada: primeiro a fila do que precisa de alguém, com a idade de cada item; depois o mês por cliente em tabela; os contadores viram uma linha. Serve para mostrar densidade, tipografia e hierarquia; não é para copiar pixel a pixel.

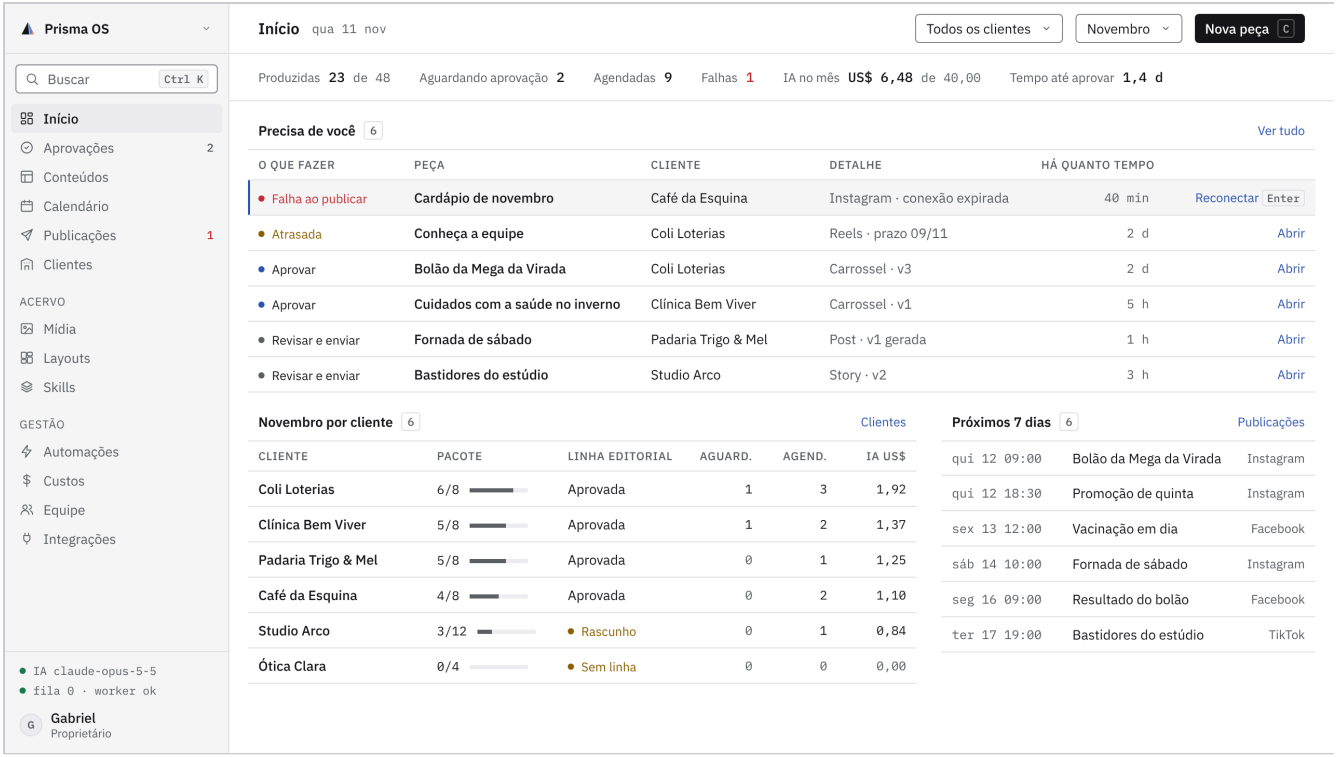


Figura 4. A mesma tela com os tokens do tema claro: só as variáveis mudam. Recomendação: entregar os dois temas, seguir a preferência do sistema dentro do app e usar o claro nas páginas que o cliente abre (aprovar e relatório).

4 Tela por tela

TELA	HOJE	COMO FICA
Estrutura	Barra lateral de 240 px com 15 itens em pílula; título de 36 px no corpo da página.	Barra lateral de 216 px, linhas de 28 px, contador ao lado de Aprovações e de Publicações (falha em vermelho), grupo Gestão no fim. Barra de topo com título, filtros e ação principal. Atalhos: `C` cria peça, `G` seguido de letra navega.
Início	Seis contadores, dois cartões de lista, painel do mês, atividade, produção e tabela por cliente.	"Precisa de você": uma fila única com aprovar, revisar, falhas e atrasos, ordenada pelo que está parado há mais tempo. Depois o mês por cliente em tabela e os próximos 7 dias. Contadores em uma linha.
Listas	Conteúdos, Clientes, Publicações, Aprovações, Custos e Histórico, cada uma com tabela dentro de cartão; em Clientes, o formulário "Novo cliente" ocupa o topo.	Tabela de borda a borda com o componente único. Busca, filtros que aplicam ao mudar e seleção em lote com barra de ações. "Novo cliente" vira botão com diálogo.
Página do cliente	Sete abas; na Visão geral, "Primeiros passos" em grade e formulários de dados e de teto de IA na coluna direita.	Cabeçalho compacto com segmento, situação e pacote em linha. "Primeiros passos" vira lista que se recolhe e some quando completa. Dados e limites vão para uma aba Configurações.
Perfil da marca e Criar	Duas colunas de áreas de texto vazias de 96 px; cartões numerados; barra de salvar sempre visível.	Uma coluna de até 640 px, grupos com título à esquerda, áreas de texto que crescem. Barra de salvar só quando há alteração.
Peça e editor	Prévia sobre fundo quase preto, dentro de cartão, com caixas com borda dentro do cartão.	Prévia sobre cinza neutro médio, para julgar a cor da arte. Propriedades à direita em seções recolhíveis, sem caixa dentro de caixa. Baixar, copiar legenda e duplicar em um menu.
Calendário	Grade mensal com células de 112 px, quase todas vazias.	Alternar entre Mês e Lista. Lista agrupada por dia como padrão enquanto houver poucas pautas; grade com célula compacta.
Custos	Cinco contadores e quatro tabelas em cartões; tabela cortada a 1440 px; casas decimais diferentes.	Uma tabela com abas (por cliente, por skill, execuções, tarifas). Valores sempre com duas casas, quatro só no detalhe da execução.

TELA	HOJE	COMO FICA
Integrações	Três cartões altos com todos os campos abertos.	Lista de integrações com estado e última verificação; a configuração abre em painel ao clicar.
Login	Frase de vitrine e print do produto ao lado do formulário.	Formulário centralizado: logotipo, e-mail, senha.
Páginas do cliente	Aprovação em fundo quase preto, com raios próprios.	Aprovação e relatório em tema claro, com os mesmos tokens e componentes. (O e-mail em HTML entra na Fase 4, item A10.)
Celular	Gaveta de navegação e cartões no lugar da tabela já funcionam.	Manter. Depois da compactação, conferir alvos de toque de 44 px e ausência de rolagem horizontal de 360 a 1440 px.

5 Automações do produto

5.1 O que já roda sozinho

TAREFA	QUANDO	ONDE
Geração de peça · `content.generate`	ao pedir uma peça ou aprovar a linha editorial	`jobs/generate-content.ts`
Publicação · `social.publish`	na hora agendada, com nova tentativa	`services/publications.ts`
Métricas · `social.metrics`	uma vez, 24 h depois de publicar; TikTok fora	`services/publications.ts`
Webhook · `webhook.deliver`	em cada um dos 5 eventos	`services/webhooks.ts`
E-mail · `email.send`	pedido de aprovação, decisão, publicação que falhou	`services/notifications.ts`
Limpeza · `runMaintenance`	de hora em hora, dentro do laço do worker	`services/maintenance.ts`
Backup	todo dia às 03:17, por cron, só na instalação em servidor	`servidor/backup.sh`

5.2 A base que falta

1. **Agendador recorrente.** A fila só conhece tarefa avulsa com `run_after`. Criar a tabela `schedules` (regra, escopo, recorrência, fuso `America/Sao_Paulo`, `next_run_at`) e, em `runWorkerLoop`, enfileirar o que venceu com chave de idempotência `regra:período`. O worker continua sendo o único processo; nada de cron externo.
2. **Regras com liga e desliga.** Tabela `automation_rules` por agência e por cliente, com parâmetros (dias, horário). Tudo que gasta IA ou publica nasce desligado. Mudança de regra entra na auditoria.
3. **Telas Automações e Fila.** Cada regra com estado, última execução, próxima execução e resultado. Na aba Fila, as tarefas de `jobs` com erro e botão de reprocessar. Hoje a fila não tem tela.
4. **Contato do cliente.** A tabela `clients` não tem e-mail nem WhatsApp; os links saem por `wa.me` sem número. Sem esse cadastro, nenhuma automação fala com o cliente.

5.3 Lista priorizada

T é tamanho relativo: P cabe em uma sessão curta, M em uma sessão, G em mais de uma.

#	AUTOMAÇÃO	HOJE	PROPOSTA	T	FASE
A1	**Falha de geração**	`markGenerationFailed` só muda o estado da peça; ninguém é avisado.	Aviso no sino e por e-mail a quem pediu; evento `content.failed`.	P	3
A2	**Teto de IA**	O campo "Alertar ao atingir (%)" existe, mas `assertWithinBudget` devolve `alert` e os três chamadores ignoram. Só muda a cor do percentual em Custos.	Aviso aos administradores ao cruzar o limite e ao chegar a 100%, uma vez por cliente e por mês.	P	3
A3	**Aprovação parada**	Não há lembrete. O link do cliente vence em 14 dias sem aviso.	Lembrete à equipe depois de N dias; ao cliente, quando houver contato; aviso 2 dias antes de o link vencer.	P	3
A4	**Prazos**	A peça tem responsável e prazo e ganha a etiqueta "Atrasada", só na tela.	Resumo diário às 8 h para cada pessoa: o que vence hoje e o que atrasou.	P	3

#	AUTOMAÇÃO	HOJE	PROPOSTA	T	FASE
A5	**Tarefa que falhou de vez**	Falha de e-mail ou webhook fica no registro e, no caso do webhook, em Integrações.	Aviso ao administrador e reprocessar pela aba Fila.	P	3
A6	**Google Drive**	"Importar agora": 25 arquivos por clique, dentro da requisição.	Tarefa `drive.sync` por cliente toda noite, na fila, com aviso quando chegam arquivos.	M	4
A7	**Métricas**	Uma leitura; cada leitura nova substitui a anterior em `publications.metrics`.	Leituras em 1, 7 e 30 dias, guardadas em série. Alimenta o relatório.	M	4
A8	**Saúde das conexões**	O token só é testado na hora de publicar; o problema aparece como publicação que falhou.	Verificação diária de cada conta e 24 h antes de cada agendamento; aviso para reconectar.	M	4
A9	**Resumo semanal**	Não existe.	E-mail de segunda-feira: produzido × contratado por cliente, pendências, publicações da semana.	P	4
A10	**E-mails**	Texto puro (`sendMail` só com `text`).	HTML simples com miniatura da peça e botão; versão em texto mantida.	P	4
A11	**Linha editorial do mês seguinte**	Alguém clica em propor, cliente por cliente.	No dia 20, rascunho para cada cliente com pacote. A equipe revisa e aprova como hoje.	M	5
A12	**Relatório do mês**	Montado e enviado à mão.	No dia 1, link do mês anterior pronto e aviso ao responsável. Envio direto ao cliente só se a regra estiver ligada.	P	5
A13	**Pautas recorrentes**	Não existe.	Regra por cliente (toda semana, todo mês) que cria a pauta na linha editorial.	M	5
A14	**Agendar ao aprovar**	Aprovar e publicar são passos manuais separados, por decisão de projeto.	Regra por cliente: peça aprovada com data de pauta vira publicação agendada, cancelável, com aviso.	M	5
A15	**Ajuste pedido pelo cliente**	A reprovação vira sugestão na memória do cliente.	A IA prepara a versão seguinte com o comentário e deixa em revisão. Ninguém envia sem olhar.	M	5
A16	**Webhooks e API de entrada**	5 eventos de saída; nenhuma entrada.	Mais eventos (`content.failed`, `budget.threshold`, `report.ready`, `drive.imported`, `task.overdue`) e API com token para criar pauta ou pedido a partir do n8n ou de um formulário.	G	5

Para depois, se fizer sentido: envio do link de aprovação pelo WhatsApp oficial (API da Meta: exige número comercial e modelos de mensagem aprovados, e é cobrada) e sugestão de melhor horário a partir do histórico de métricas (depende de A7).

6 Automações de engenharia

#	ITEM	HOJE	PROPOSTA	FASE
E1	**CI**	Não existe `.github/`. A conferência antes do push é manual (lista em `docs/design/PLANO-PROXIMA-ETAPA.md`).	GitHub Actions em todo PR: `typecheck`, `lint`, `test` com serviço PostgreSQL 16, `build`, `test:e2e`. Relatório do Playwright como artefato.	0
E2	**Branch e PR**	Único branch `claude/session-vagl59`. 31 commits, todos com autor "Claude"; média de 29 arquivos e cerca de 1.170 linhas inseridas por commit, sem contar o `package-lock.json`; o maior tem 8.886.	`main` protegido, um PR por fase com descrição e capturas, commits pequenos.	0
E3	**Formatação**	Sem Prettier. 7,8% das linhas passam de 120 colunas, 172 passam de 200, a maior tem 622.	Prettier com `prettier-plugin-tailwindcss`, `lint-staged` no pre-commit, `format:check` no CI. Um commit só de formatação, listado em `.git-blame-ignore-revs`.	0

#	ITEM	HOJE	PROPOSTA	FASE
E4	**Dependências**	Nada automatizado. `@fontsource/newsreader` e `@fontsource/schibsted-grotesk` estão no `package.json` sem uso.	Dependabot ou Renovate semanal e agrupado, `npm audit --omit=dev` no CI, `knip` para arquivos, exports e pacotes sem uso.	0
E5	**Dados de demonstração**	Não há script de dados. No Codespace, as telas estão quase vazias, e é assim que o visual vem sendo avaliado.	`npm run seed:demo`: 8 clientes e cerca de 60 peças em todos os estados, com publicações e custos. Usado no desenvolvimento, nas capturas e nos testes visuais.	0
E6	**Capturas e regressão visual**	`docs/img` é atualizado à mão.	Playwright `toHaveScreenshot()` em 10 telas, desktop e celular, com o seed. Script que regenera `docs/img`.	0
E7	**Acessibilidade**	Conferida à mão.	`@axe-core/playwright` nas telas principais; violação séria falha o CI.	2
E8	**Imagem Docker**	Um estágio, com dependências de desenvolvimento e código-fonte. Worker e migrations rodam com `tsx` em produção. Só o Postgres tem `healthcheck`.	Multi-estágio, Next em modo `standalone`, worker e migrations compilados, `HEALTHCHECK` em web e worker.	6
E9	**Deploy**	`sudo bash servidor/atualizar.sh` no servidor: backup, `git pull` e build no próprio VPS. `aguardar_site` aceita resposta 503 como "no ar" e não há volta à versão anterior.	Imagem construída no CI e publicada no GHCR por tag; o servidor só baixa. Deploy por `workflow_dispatch`. Se `/api/health` não responder 200, volta sozinho para a imagem anterior.	6
E10	**Monitoramento**	`/api/health` existe e ninguém consulta.	Verificação externa a cada minuto com alerta por e-mail ou Telegram. Batimento do worker e do backup em serviço de "dead man's switch". Captura de erros (Sentry ou GlitchTip) em web e worker.	6
E11	**Backup**	Diário, 14 cópias, cópia externa opcional. `config.env` e `app.env` (senhas do banco, `APP_SECRET_KEY`, chaves) vão em texto puro dentro do backup, inclusive para o destino remoto.	Cópia externa exigida pelo instalador, backup cifrado (age ou gpg) antes de sair do servidor, restauração de teste semanal com alerta.	6
E12	**Versões**	`0.1.0` desde o início, sem tags nem changelog.	release-please ou changesets; versão e commit no rodapé e em `/api/health`.	6

7 Outros pontos que um revisor sênior apontaria

- **IA e redes ainda não rodaram de verdade.** `docs/STATUS.md` registra que Anthropic, OpenAI, Meta, TikTok, YouTube e Drive foram testados contra APIs simuladas, com a conexão real pendente; no Codespace a IA aparece como não conectada. Antes de polir, vale fechar um ciclo real com um cliente piloto: gerar, aprovar e publicar.
- **Documentação com cara de relatório de sessão, e desatualizada.** `docs/STATUS.md` diz que módulos futuros aparecem desabilitados no menu; `shell/sidebar.tsx` não tem esses itens. `docs/design/PLANO-PROXIMA-ETAPA.md` é a lista de trabalho de uma sessão. Trocar por `CHANGELOG.md` e um `README` curto; `docs/DECISIONS.md` já está no formato certo.
- **Arquivos grandes.** `services/content.ts` (851 linhas), `services/publications.ts` (587), `render/kit.tsx` (570), `clientes/[id]/page.tsx` (551), `conteudos/[id]/editor.tsx` (540). Dividir por responsabilidade quando a fase tocar neles.
- **`src/components/ui.tsx` mistura componentes e formatadores** (`formatDateTime`, `formatUsd`). Separar em `src/components/ui/` e `src/lib/format.ts`.
- **Rotas de API em dois idiomas.** `clients`, `contents` e `media` ao lado de `agenda`, `busca`, `conexoes` e `publico`. A decisão 14 de `docs/DECISIONS.md` diz inglês.
- **Sem Content-Security-Policy** em `next.config.ts`. Os outros cabeçalhos de segurança estão lá e o HSTS vem do Caddy.

8 Plano por fases

Cada fase é um PR (a Fase 2, dois ou três). "Pronto quando" é o critério de aceite.

FASE	ENTREGA	PRONTO QUANDO
0 · Base	E1 a E6.	Um PR de teste roda o CI inteiro em verde. `npm run seed:demo` popula um banco vazio. Existe a pasta de capturas do "antes", gerada pelo Playwright.
1 · Fundamentos visuais	Tokens e fontes em `globals.css`. `src/components/ui/` com botão, campo, seleção, campo com busca, estado, aviso, tabela, barra de página, seção, par rótulo e valor, linha de vazio, abas, diálogo, aviso de resultado e tecla. Barra lateral e barra de topo. Temas claro e escuro. `docs/design/SISTEMA.md`. `scripts/check-design.mjs`.	Nenhuma ocorrência do que a trava proíbe. Testes e E2E verdes sem mudar comportamento. PR com capturas antes e depois de todas as telas.
2 · Telas	2a: Início e listas. 2b: página do cliente e formulários. 2c: peça e editor, calendário, login e páginas do cliente. Tudo conforme a seção 4. E7.	axe sem violação séria. Sem rolagem horizontal de 360 a 1440 px. Uma captura por tela com o seed.
3 · Automações I	Base da seção 5.2 e A1 a A5.	A tela Automações mostra última e próxima execução de cada regra. Cada automação tem teste de integração com relógio simulado e não dispara duas vezes no mesmo período.
4 · Automações II	A6 a A10.	O mesmo critério da Fase 3. Série de métricas visível no relatório.
5 · Automações III	A11 a A16, todas desligadas por padrão.	O mesmo critério. O custo de IA de cada automação aparece em Custos com o nome da regra e respeita o teto do cliente.
6 · Operação	E8 a E12.	Deploy por um clique, com a volta automática testada derrubando o health de propósito. Uma restauração de teste concluída. Um alerta de queda recebido no canal escolhido.

9 Para o Claude Code

9.1 Regras

- Uma fase por PR. Redesenho não muda comportamento; automação não muda visual.
- Antes de programar: ler `AGENTS.md` e este documento, listar os arquivos que vai tocar e esperar a confirmação.
- `npm run typecheck`, `npm run lint`, `npm test` e `npm run test:e2e` verdes antes de cada commit. Teste só muda quando o redesenho muda seletor ou texto, e o PR explica.
- `docs/design/DESIGN-referencia.md` deixa de valer. A referência é a seção 3 e, depois da Fase 1, `docs/design/SISTEMA.md`.
- Nada simulado: sem dado, botão ou integração de mentira. O projeto já segue esse princípio.
- Interface em português do Brasil. Texto existente só muda onde a seção 4 pede.
- Migration nova para cada mudança de banco; nunca editar as existentes.
- Automação que gasta IA ou publica nasce desligada, é ligada por cliente e registra na auditoria.
- Perguntar antes de mudar cor ou logotipo da marca, remover funcionalidade ou trocar dependência de produção.
- PR com capturas antes e depois, desktop e celular, e a lista do que ficou de fora.

9.2 Pedidos prontos

Salve este PDF no repositório como `docs/briefing-melhorias.pdf` e use um pedido por vez, na ordem. Cada um vira um PR.

FASE 0

```
Leia AGENTS.md e docs/briefing-melhorias.pdf. Execute somente a Fase 0 (itens E1 a E6 da seção 6). Antes de escrever código, mostre o plano com a lista de arquivos e espere minha confirmação. Crie o branch main, abra o PR contra ele e confirme que o CI rodou.
```

FASE 1

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 2, 3 e 9. Execute somente a Fase 1: tokens, fontes, componentes de src/components/ui, barra lateral, barra de topo, temas claro e escuro, docs/design/SISTEMA.md e scripts/check-design.mjs. Não altere comportamento nem textos. Mostre o plano antes. No PR, inclua capturas antes e depois de todas as telas, geradas pelo Playwright com o seed.
```

FASE 2A

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 3, 4 e 9. Execute somente a Fase 2a: Início e listas. Use a Figura 3 como direção de densidade e hierarquia, não como layout para copiar. A fila "Precisa de você" deve usar dados reais de tarefas, aprovações, publicações com falha e prazos. Mostre o plano antes.
```

FASE 2B

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 3, 4 e 9. Execute somente a Fase 2b: página do cliente, Perfil da marca, Criar e Integrações. Formulários em uma coluna, áreas de texto que crescem e barra de salvar só quando houver alteração. Mostre o plano antes.
```

FASE 2C

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 3, 4 e 9. Execute somente a Fase 2c: peça e editor, calendário, login e as páginas do cliente (aprovar e relatório) em tema claro. Inclua o teste de acessibilidade com axe (item E7). Mostre o plano antes.
```

FASE 3

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 5 e 9. Execute somente a Fase 3: agendador recorrente, telas Automações e Fila, contato do cliente e as automações A1 a A5. Mostre o desenho das tabelas novas antes de criar a migration. Cada automação precisa de teste de integração com relógio simulado e chave de idempotência por período.
```

FASE 4

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 5 e 9. Execute somente a Fase 4: automações A6 a A10, usando o agendador e as regras da Fase 3. Mostre o plano e as tabelas novas antes. Mesmos critérios de teste da Fase 3.
```

FASE 5

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 5 e 9. Execute somente a Fase 5: automações A11 a A16, todas desligadas por padrão e ligadas por cliente. As que usam IA respeitam o teto do cliente e aparecem em Custos com o nome da regra. Mostre o plano antes.
```

FASE 6

```
Leia AGENTS.md e docs/briefing-melhorias.pdf, seções 6 e 9. Execute somente a Fase 6: itens E8 a E12. Não altere o servidor em produção sem minha confirmação; teste o deploy e a volta automática primeiro em um ambiente de teste. Mostre o plano antes.
```